Bulletproofs: inner-product argument

October 29, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1 Introduction

2 Zero-knowledge multiplication

3 Inner-product argument

Introduction

Bulletproofs: just some basic linear algebra



Bulletproofs: Motivation

- **Bulletproofs:** zero-knowledge proofs with logarithmic proof size

Bulletproofs: Motivation

- **Bulletproofs:** zero-knowledge proofs with logarithmic proof size
- No trusted setup, just some basic linear algebra

Bulletproofs: Motivation

- **Bulletproofs:** zero-knowledge proofs with logarithmic proof size
- No trusted setup, just some basic linear algebra
- Straightforward cryptographic assumption: discrete logarithm without bilinear pairings or other advanced assumptions

Bulletproofs: Motivation

- **Bulletproofs:** zero-knowledge proofs with logarithmic proof size
- No trusted setup, just some basic linear algebra
- Straightforward cryptographic assumption: discrete logarithm without bilinear pairings or other advanced assumptions
- Originally developed for efficient range proofs in confidential transactions

Bulletproofs: Motivation

- **Bulletproofs:** zero-knowledge proofs with logarithmic proof size
- No trusted setup, just some basic linear algebra
- Straightforward cryptographic assumption: discrete logarithm without bilinear pairings or other advanced assumptions
- Originally developed for efficient range proofs in confidential transactions
- Applicable for proving arithmetic circuits satisfiability (e.g., R1CS)

Bulletproofs: Motivation

- **Bulletproofs:** zero-knowledge proofs with logarithmic proof size
- No trusted setup, just some basic linear algebra
- Straightforward cryptographic assumption: discrete logarithm without bilinear pairings or other advanced assumptions
- Originally developed for efficient range proofs in confidential transactions
- Applicable for proving arithmetic circuits satisfiability (e.g., R1CS)
- Efficient polynomial commitment scheme could be derived (e.g., *IPA* polynomial commitment)

Bulletproofs: Motivation

- **Bulletproofs:** zero-knowledge proofs with logarithmic proof size
- No trusted setup, just some basic linear algebra
- Straightforward cryptographic assumption: discrete logarithm without bilinear pairings or other advanced assumptions
- Originally developed for efficient range proofs in confidential transactions
- Applicable for proving arithmetic circuits satisfiability (e.g., R1CS)
- Efficient polynomial commitment scheme could be derived (e.g., *IPA* polynomial commitment)
- Heart of *bulletproofs* – **inner-product argument**

Bulletproofs: Building blocks

- Zero-knowledge multiplication protocol **zk-mul**

Bulletproofs: Building blocks

- Zero-knowledge multiplication protocol **zk-mul**
- Inner-product argument **IPA**

Bulletproofs: Building blocks

- Zero-knowledge multiplication protocol **zk-mul**
- Inner-product argument **IPA**
- Application: *IPA* polynomial commitment scheme
- Application: range proofs
- Application: arithmetic circuits satisfiability

Preliminaries

- $\mathbb{G}$ – cyclic group of prime order p (written additively) where **DLog** assumption holds

Preliminaries

- $\mathbb{G}$ – cyclic group of prime order p (written additively) where **DLog** assumption holds
- $G, B \in \mathbb{G}$ – independent generators
- $\mathbf{G} = (G_1, \ldots, G_n), \mathbf{H} = (H_1, \ldots, H_n) \in \mathbb{G}^n$ – vectors of generators with mutually unknown discrete log relations

Preliminaries

- $\mathbb{G}$ – cyclic group of prime order p (written additively) where **DLog** assumption holds
- $G, B \in \mathbb{G}$ – independent generators
- $\mathbf{G} = (G_1, \ldots, G_n), \mathbf{H} = (H_1, \ldots, H_n) \in \mathbb{G}^n$ – vectors of generators with mutually unknown discrete log relations
- $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^{n} a_i b_i$ – inner product of scalar vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_p^n$
- $\langle \mathbf{a}, \mathbf{G} \rangle = \sum_{i=1}^{n} [a_i] G_i$ – inner product of a scalar vector $\mathbf{a} \in \mathbb{F}_p^n$ with a vector of generators $\mathbf{G} \in \mathbb{G}^n$

Preliminaries

- $\mathbb{G}$ – cyclic group of prime order p (written additively) where **DLog** assumption holds
- $G, B \in \mathbb{G}$ – independent generators
- $\mathbf{G} = (G_1, \ldots, G_n), \mathbf{H} = (H_1, \ldots, H_n) \in \mathbb{G}^n$ – vectors of generators with mutually unknown discrete log relations
- $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^{n} a_i b_i$ – inner product of scalar vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_p^n$
- $\langle \mathbf{a}, \mathbf{G} \rangle = \sum_{i=1}^{n} [a_i] G_i$ – inner product of a scalar vector $\mathbf{a} \in \mathbb{F}_p^n$ with a vector of generators $\mathbf{G} \in \mathbb{G}^n$
- $\mathbf{k}^n = (1, k, k^2, \ldots, k^{n-1})$; for example, $\mathbf{0}^n, \mathbf{1}^n$ are the vectors of zeros and ones, while $\mathbf{2}^n = (1, 2, 4, \ldots, 2^{n-1})$

Zero-knowledge multiplication

Zero-knowledge multiplication

Goal: Prove knowledge of $a, b \in \mathbb{F}_p$ such that $c = ab$ without revealing a, b, c, i.e. consider the relation:

$$\mathcal{R}_{abc} = \{(\perp; c, a, b) : c = ab\}$$

Zero-knowledge multiplication

Goal: Prove knowledge of $a, b \in \mathbb{F}_p$ such that $c = ab$ without revealing a, b, c, i.e. consider the relation:

$$\mathcal{R}_{abc} = \{(\perp; c, a, b) : c = ab\}$$

Curious reader could argue that this is relatively simple problem as we have Σ-protocols framework, especially Chaum-Pedersen protocol for DH-triplets, e.g. we could prove slightly modified relation:

$$\mathcal{R}'_{abc} = \{(P, Q_a, Q_b, Q_c \in \mathbb{G}; a, b) : Q_c = [a]Q_b, Q_a = [a]P, Q_b = [b]P\}$$

Zero-knowledge multiplication

Goal: Prove knowledge of $a, b \in \mathbb{F}_p$ such that $c = ab$ without revealing a, b, c, i.e. consider the relation:

$$\mathcal{R}_{abc} = \{(\perp; c, a, b) : c = ab\}$$

Curious reader could argue that this is relatively simple problem as we have Σ-protocols framework, especially Chaum-Pedersen protocol for DH-triplets, e.g. we could prove slightly modified relation:

$$\mathcal{R}'_{abc} = \{(P, Q_a, Q_b, Q_c \in \mathbb{G}; a, b) : Q_c = [a]Q_b, Q_a = [a]P, Q_b = [b]P\}$$

Problem

Prover does not hide a, b, c values so that adversary could potentially learn them, especially if they are small or have non-uniform distribution.

Multiplication of committed values

Solution

Use Pedersen commitments to bind to a, b, c values.

Multiplication of committed values

Solution

Use Pedersen commitments to bind to a, b, c values.

$$\mathcal{R}_{abc} = \left\{ \begin{array}{l} (G, H, B, A, T_0; a, b, \alpha, \tau_0) : \\ A = [a]G + [b]H + [\alpha]B \wedge \\ T_0 = [ab]G + [\tau_0]B \end{array} \right\}$$

Here A is a binding Pedersen commitment to both a and b while T_0 is a binding Pedersen commitment to their product ab.

Multiplication of committed values

Solution

Use Pedersen commitments to bind to a, b, c values.

$$\mathcal{R}_{abc} = \left\{ \begin{array}{l} (G, H, B, A, T_0; a, b, \alpha, \tau_0) : \\ A = [a]G + [b]H + [\alpha]B \wedge \\ T_0 = [ab]G + [\tau_0]B \end{array} \right\}$$

Here A is a binding Pedersen commitment to both a and b while T_0 is a binding Pedersen commitment to their product ab.

Problem

How to prove that in zero-knowledge?

Zero-knowledge polynomial multiplication

In order to provide zero-knowledge proof of $\mathcal{R}_{abc}$ we bring out polynomials!

$$l(x) = a + s_L x$$

$$r(x) = b + s_R x$$

$$t(x) = l(x)r(x) = ab + (as_R + bs_L)x + s_L s_R x^2 = t_0 + t_1 x + t_2 x^2$$

Zero-knowledge polynomial multiplication

In order to provide zero-knowledge proof of $\mathcal{R}_{abc}$ we bring out polynomials!

$$l(x) = a + s_L x$$
$$r(x) = b + s_R x$$
$$t(x) = l(x)r(x) = ab + (as_R + bs_L)x + s_L s_R x^2 = t_0 + t_1 x + t_2 x^2$$

What we have now:

- $s_L, s_R \in \mathbb{F}_p$ are blinding factors

Zero-knowledge polynomial multiplication

In order to provide zero-knowledge proof of $\mathcal{R}_{abc}$ we bring out polynomials!

$$l(x) = a + s_L x$$
$$r(x) = b + s_R x$$
$$t(x) = l(x)r(x) = ab + (as_R + bs_L)x + s_L s_R x^2 = t_0 + t_1 x + t_2 x^2$$

What we have now:

- $s_L, s_R \in \mathbb{F}_p$ are blinding factors
- $l(x)$ is a linear polynomial hiding value a
- $r(x)$ is a linear polynomial hiding value b

Zero-knowledge polynomial multiplication

In order to provide zero-knowledge proof of $\mathcal{R}_{abc}$ we bring out polynomials!

$$l(x) = a + s_L x$$
$$r(x) = b + s_R x$$
$$t(x) = l(x)r(x) = ab + (as_R + bs_L)x + s_L s_R x^2 = t_0 + t_1 x + t_2 x^2$$

What we have now:

- $s_L, s_R \in \mathbb{F}_p$ are blinding factors
- $l(x)$ is a linear polynomial hiding value a
- $r(x)$ is a linear polynomial hiding value b
- $t(x)$ is a quadratic polynomial, constant term is the product ab, the product we typically want to prove knowledge of.

Zero-knowledge polynomial multiplication

Idea

If $l(x)r(x) = t(x)$ then with high probability for random $u \leftarrow\$ \mathbb{F}_p$ we have $l(u)r(u) = t(u)$ due to *Schwartz-Zippel lemma*

Zero-knowledge polynomial multiplication

Idea

If $l(x)r(x) = t(x)$ then with high probability for random $u \leftarrow\$ \mathbb{F}_p$ we have $l(u)r(u) = t(u)$ due to *Schwartz-Zippel lemma*

Now we can build a zero-knowledge protocol **zk-mul** (Π_{mul}) for proving a product of degree-one polynomials $l(x)r(x) = t(x)$, given generators $G, H, B \in \mathbb{G}$ with unknown discrete log relations:

- Prover draws $\alpha, \beta, \tau_0, \tau_1, \tau_2 \leftarrow\$ \mathbb{F}_p$, computes and sends to $\mathcal{V}$ commitments to coefficients of $l(x), r(x), t(x)$:

$$A = [a]G + [b]H + [\alpha]B \qquad T_0 = [ab]G + [\tau_0]B$$
$$S = [s_L]G + [s_R]H + [\beta]B \qquad T_1 = [as_R + bs_L]G + [\tau_1]B$$
$$T_2 = [s_L s_R]G + [\tau_2]B$$

  Here A – commitment to constant terms, S – commitment to degree-one coefficients of $l(x), r(x)$, T_i for $i \in \{0, 1, 2\}$ – commitments to coefficients of $t(x)$.

Zero-knowledge polynomial multiplication

- Verifier draws random challenge $u \leftarrow\$ \mathbb{F}_p$ and sends it to prover

Zero-knowledge polynomial multiplication

- Verifier draws random challenge $u \leftarrow\$ \mathbb{F}_p$ and sends it to prover
- Prover evaluates and sends to Verifier $(l_u, r_u, t_u, \alpha_u, \tau_u)$:

$$l_u = l(u),\ r_u = r(u),\ t_u = t(u) = l_u \cdot r_u,$$
$$\alpha_u = \alpha + \beta u,\ \tau_u = \tau_0 + \tau_1 u + \tau_2 u^2$$

Zero-knowledge polynomial multiplication

- Verifier draws random challenge $u \leftarrow\$ \mathbb{F}_p$ and sends it to prover
- Prover evaluates and sends to Verifier $(l_u, r_u, t_u, \alpha_u, \tau_u)$:

$$l_u = l(u),\ r_u = r(u),\ t_u = t(u) = l_u \cdot r_u,$$
$$\alpha_u = \alpha + \beta u,\ \tau_u = \tau_0 + \tau_1 u + \tau_2 u^2$$

- Verifier checks:

$$A + [u]S \stackrel{?}{=} [l_u]G + [r_u]H + [\alpha_u]B$$
$$[t_u]G + [\tau_u]B \stackrel{?}{=} T_0 + [u]T_1 + [u^2]T_2$$
$$t_u \stackrel{?}{=} l_u r_u$$

Zero-knowledge numbers multiplication

Now we could easily tweak our **zk-mul** protocol to prove relation

$$\mathcal{R}_{abc} = \left\{ \begin{array}{l} (G, H, B, A, T_0; a, b, \alpha, \tau_0) : \\ A = [a]G + [b]H + [\alpha]B \wedge \\ T_0 = [ab]G + [\tau_0]B \end{array} \right\}$$

in zero-knowledge:

Zero-knowledge numbers multiplication

Now we could easily tweak our **zk-mul** protocol to prove relation

$$\mathcal{R}_{abc} = \left\{ \begin{array}{l} (G, H, B, A, T_0; a, b, \alpha, \tau_0) : \\ A = [a]G + [b]H + [\alpha]B \wedge \\ T_0 = [ab]G + [\tau_0]B \end{array} \right\}$$

in zero-knowledge: the prover draws random $s_L, s_R \leftarrow\!\!\$\ \mathbb{F}_p$, and the parties just use the prescribed commitment A as a commitment to constant terms of $l(x), r(x)$ and T_0 as a commitment to constant coefficient of $t(x)$.

zk-mul protocol: Security

Theorem

Zero-knowledge polynomial multiplication protocol **zk-mul** is perfect complete, special sound and perfect honest-verifier zero-knowledge.

zk-mul protocol: Security

Theorem

Zero-knowledge polynomial multiplication protocol **zk-mul** is perfect complete, special sound and perfect honest-verifier zero-knowledge.

Here we briefly show the completeness:

- First check:

$$A + [u]S \stackrel{?}{=} [l_u]G + [r_u]H + [\alpha_u]B$$

$$LHS = [a]G + [b]H + [\alpha]B + [us_L]G + [us_R]H + [u\beta]B$$

$$RHS = [a + us_L]G + [b + us_R]H + [\alpha + u\beta]B =$$

$$= [a]G + [b]H + [\alpha]B + [us_L]G + [us_R]H + [u\beta]B$$

As $LHS = RHS$ the check is satisfied.

zk-mul protocol: Security

- Second check:

$$[t_u]G + [\tau_u]B \stackrel{?}{=} T_0 + [u]T_1 + [u^2]T_2$$

$$LHS = [ab + t_1 u + t_2 u^2]G + [\tau_0 + \tau_1 u + \tau_2 u^2]B$$

$$RHS = [ab]G + [\tau_0]B + [u]([t_1]G + [\tau_1]B) +$$

$$+ [u^2]([t_2]G + [\tau_2]B)$$

As $LHS = RHS$ the check is satisfied.

zk-mul protocol: Security

- Second check:

$$[t_u]G + [\tau_u]B \stackrel{?}{=} T_0 + [u]T_1 + [u^2]T_2$$

$$LHS = [ab + t_1 u + t_2 u^2]G + [\tau_0 + \tau_1 u + \tau_2 u^2]B$$

$$RHS = [ab]G + [\tau_0]B + [u]([t_1]G + [\tau_1]B) +$$

$$+ [u^2]([t_2]G + [\tau_2]B)$$

  As $LHS = RHS$ the check is satisfied.

- The third check is $t_u = l_u r_u$ holds by definition

zk-mul protocol: Security

- Second check:

$$[t_u]G + [\tau_u]B \stackrel{?}{=} T_0 + [u]T_1 + [u^2]T_2$$

$$LHS = [ab + t_1 u + t_2 u^2]G + [\tau_0 + \tau_1 u + \tau_2 u^2]B$$

$$RHS = [ab]G + [\tau_0]B + [u]([t_1]G + [\tau_1]B) +$$

$$+ [u^2]([t_2]G + [\tau_2]B)$$

  As $LHS = RHS$ the check is satisfied.

- The third check is $t_u = l_u r_u$ holds by definition

Intuitively **zk-mul** protocol is also *zero-knowledge* as it is easy to simulate every step of the honest prover. *Special soundness* holds as it's easy to build an extractor similar to Okamoto's protocol extractor for extracting openings of Pedersen commitments.

zk-mul: inner-product version

We could easily generalize **zk-mul** protocol to prove $t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle$ for polynomials with vector coefficients $\mathbf{l}(x), \mathbf{r}(x) \in \mathbb{F}_p^n[x]$.

zk-mul: inner-product version

We could easily generalize **zk-mul** protocol to prove $t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle$ for polynomials with vector coefficients $\mathbf{l}(x), \mathbf{r}(x) \in \mathbb{F}_p^n[x]$. Specifically using generalized **zk-mul** protocol Π_{zkip} we could also prove in zero-knowledge that inner-product relation holds for vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_p^n$:

$$\mathcal{R}_{zkip} = \left\{ \begin{array}{r} (\mathbf{G}, \mathbf{H}, G, B, A, V; \mathbf{a}, \mathbf{b}, \alpha, \gamma) : A = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\alpha]B, \\ V = [\langle \mathbf{a}, \mathbf{b} \rangle]G + [\gamma]B \end{array} \right\}$$

zk-mul: inner-product version

We could easily generalize **zk-mul** protocol to prove $t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle$ for polynomials with vector coefficients $\mathbf{l}(x), \mathbf{r}(x) \in \mathbb{F}_p^n[x]$. Specifically using generalized **zk-mul** protocol Π_{zkip} we could also prove in zero-knowledge that inner-product relation holds for vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_p^n$:

$$\mathcal{R}_{zkip} = \left\{ (\mathbf{G}, \mathbf{H}, G, B, A, V; \mathbf{a}, \mathbf{b}, \alpha, \gamma) : \begin{aligned} A &= \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\alpha]B, \\ V &= [\langle \mathbf{a}, \mathbf{b} \rangle]G + [\gamma]B \end{aligned} \right\}$$

Problem

Proof size is linear in n as the prover should send evaluated vectors $\mathbf{l}_u, \mathbf{r}_u$. We will address this problem in the next section: **inner-product argument**.

Inner-product argument

Motivation: Inner-product Argument

The heart of **bulletproofs** is **inner-product argument (IPA)** which allows to soundly prove inner-product relation between two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_p^n$:

$$\mathcal{R}_{ip} = \{(\mathbf{G}, \mathbf{H}, P, c; \mathbf{a}, \mathbf{b}) : P = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle \wedge \langle \mathbf{a}, \mathbf{b} \rangle = c\}$$

Motivation: Inner-product Argument

The heart of **bulletproofs** is **inner-product argument (IPA)** which allows to soundly prove inner-product relation between two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_p^n$:

$$\mathcal{R}_{ip} = \{(\mathbf{G}, \mathbf{H}, P, c; \mathbf{a}, \mathbf{b}) : P = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle \wedge \langle \mathbf{a}, \mathbf{b} \rangle = c\}$$

Firstly, let's combine statements $P = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle \wedge \langle \mathbf{a}, \mathbf{b} \rangle = c$ into a single statement by multiplying the second one by a random challenge $r \leftarrow\!\!\$\ \mathbb{F}_p$ and some orthogonal generator $B \in \mathbb{G}$, summing up:

$$\mathcal{R}'_{ip} = \{(\mathbf{G}, \mathbf{H}, Q, P'; \mathbf{a}, \mathbf{b}) : P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle]Q\}$$

where $Q = [r]B, P' = P + [cr]B = P + [c]Q$.

Compression step

Assuming $n = 2^d$ define by $\mathbf{G_{lo}} = (G_1, \ldots, G_{n/2}), \mathbf{G_{hi}} = (G_{n/2+1}, \ldots, G_n) \in \mathbb{G}^{n/2}$ – lower and higher halves of vector $\mathbf{G}$ and $\mathbf{a_{lo}} = (a_1, \ldots, a_{n/2}), \mathbf{a_{hi}} = (a_{n/2+1}, \ldots, a_n) \in \mathbb{F}_p^{n/2}$ – lower and higher halves of $\mathbf{a} \in \mathbb{F}_p^n$.

Compression step

Assuming $n = 2^d$ define by
$\mathbf{G_{lo}} = (G_1, \ldots, G_{n/2}), \mathbf{G_{hi}} = (G_{n/2+1}, \ldots, G_n) \in \mathbb{G}^{n/2}$ – lower and higher halves of vector $\mathbf{G}$ and
$\mathbf{a_{lo}} = (a_1, \ldots, a_{n/2}), \mathbf{a_{hi}} = (a_{n/2+1}, \ldots, a_n) \in \mathbb{F}_p^{n/2}$ – lower and higher halves of $\mathbf{a} \in \mathbb{F}_p^n$.

Let $u_k \in \mathbb{F}_p$ be a challenge scalar, define compressed vectors:

$$\mathbf{a}^{(k-1)} = \mathbf{a_{lo}} \cdot u_k + u_k^{-1} \cdot \mathbf{a_{hi}}$$
$$\mathbf{b}^{(k-1)} = \mathbf{b_{lo}} \cdot u_k^{-1} + u_k \cdot \mathbf{b_{hi}}$$
$$\mathbf{G}^{(k-1)} = \mathbf{G_{lo}} \cdot u_k^{-1} + u_k \cdot \mathbf{G_{hi}}$$
$$\mathbf{H}^{(k-1)} = \mathbf{H_{lo}} \cdot u_k + u_k^{-1} \cdot \mathbf{H_{hi}}$$

Compression step

Assuming $n = 2^d$ define by
$\mathbf{G_{lo}} = (G_1, \ldots, G_{n/2}), \mathbf{G_{hi}} = (G_{n/2+1}, \ldots, G_n) \in \mathbb{G}^{n/2}$ – lower and higher halves of vector $\mathbf{G}$ and
$\mathbf{a_{lo}} = (a_1, \ldots, a_{n/2}), \mathbf{a_{hi}} = (a_{n/2+1}, \ldots, a_n) \in \mathbb{F}_p^{n/2}$ – lower and higher halves of $\mathbf{a} \in \mathbb{F}_p^n$.

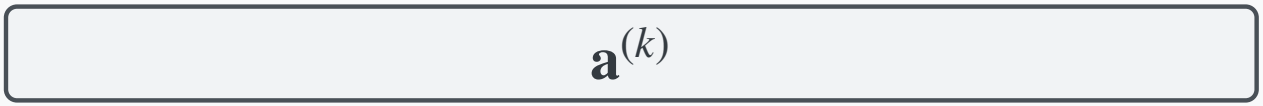
Let $u_k \in \mathbb{F}_p$ be a challenge scalar, define compressed vectors:

$$\begin{aligned}
\mathbf{a}^{(k-1)} &= \mathbf{a_{lo}} \cdot u_k + u_k^{-1} \cdot \mathbf{a_{hi}} \\
\mathbf{b}^{(k-1)} &= \mathbf{b_{lo}} \cdot u_k^{-1} + u_k \cdot \mathbf{b_{hi}} \\
\mathbf{G}^{(k-1)} &= \mathbf{G_{lo}} \cdot u_k^{-1} + u_k \cdot \mathbf{G_{hi}} \\
\mathbf{H}^{(k-1)} &= \mathbf{H_{lo}} \cdot u_k + u_k^{-1} \cdot \mathbf{H_{hi}}
\end{aligned}$$

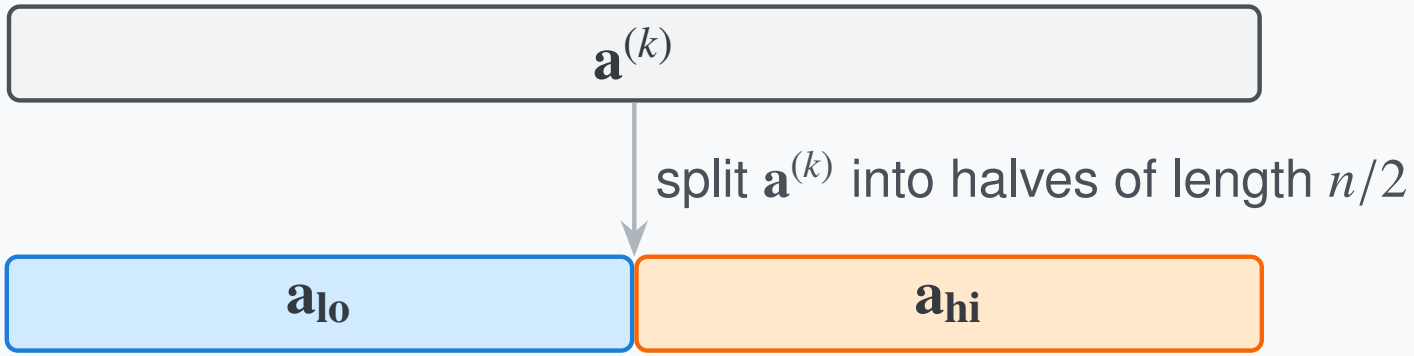
Remark

If n is not a power of two we could pad vectors with zeroes to the next power of two.

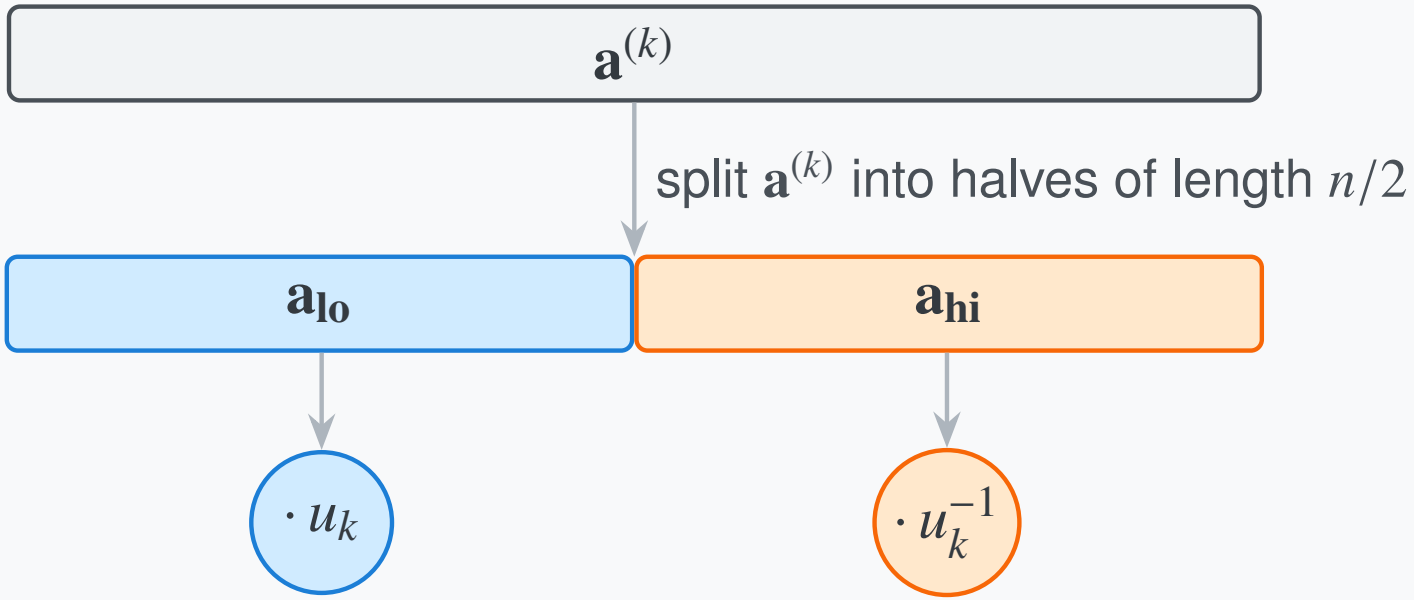
Compression step: illustration



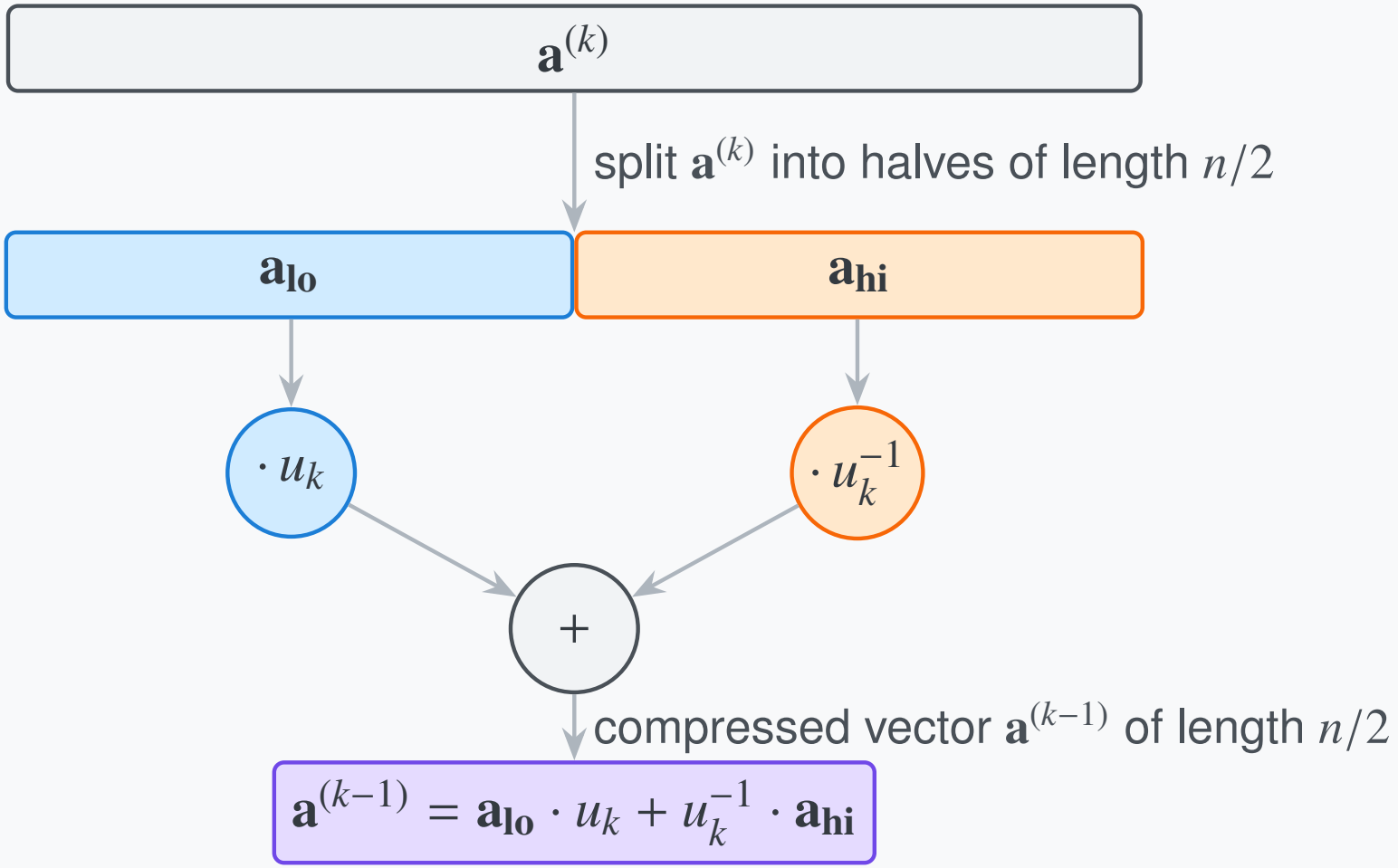
Compression step: illustration



Compression step: illustration



Compression step: illustration



Commitment compression

Define $P_k \leftarrow P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle] Q$ – current commitment to vectors $\mathbf{a}, \mathbf{b}$ and define P_{k-1} using compressed vectors to have the same form as P_k, but in new basis $(\mathbf{G}^{(k-1)}, \mathbf{H}^{(k-1)})$:

$$P_{k-1} = \langle \mathbf{a}^{(k-1)}, \mathbf{G}^{(k-1)} \rangle + \langle \mathbf{b}^{(k-1)}, \mathbf{H}^{(k-1)} \rangle + [\langle \mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)} \rangle] Q$$

Commitment compression

Define $P_k \leftarrow P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle]Q$ – current commitment to vectors $\mathbf{a}, \mathbf{b}$ and define P_{k-1} using compressed vectors to have the same form as P_k, but in new basis $(\mathbf{G}^{(k-1)}, \mathbf{H}^{(k-1)})$:

$$P_{k-1} = \langle \mathbf{a}^{(k-1)}, \mathbf{G}^{(k-1)} \rangle + \langle \mathbf{b}^{(k-1)}, \mathbf{H}^{(k-1)} \rangle + [\langle \mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)} \rangle]Q$$

Substituting compressed vectors and applying bilinearity property of inner product we get:

$$\begin{aligned} P_{k-1} = &\langle \mathbf{a_{lo}}, \mathbf{G_{lo}} \rangle + \langle \mathbf{a_{hi}}, \mathbf{G_{hi}} \rangle && + u_k^2 \langle \mathbf{a_{lo}}, \mathbf{G_{hi}} \rangle + u_k^{-2} \langle \mathbf{a_{hi}}, \mathbf{G_{lo}} \rangle + \\ &\langle \mathbf{b_{lo}}, \mathbf{H_{lo}} \rangle + \langle \mathbf{b_{hi}}, \mathbf{H_{hi}} \rangle && + u_k^2 \langle \mathbf{b_{hi}}, \mathbf{H_{lo}} \rangle + u_k^{-2} \langle \mathbf{b_{lo}}, \mathbf{H_{hi}} \rangle + \\ &[\langle \mathbf{a_{lo}}, \mathbf{b_{lo}} \rangle + \langle \mathbf{a_{hi}}, \mathbf{b_{hi}} \rangle]Q && + [u_k^2 \langle \mathbf{a_{lo}}, \mathbf{b_{hi}} \rangle + u_k^{-2} \langle \mathbf{a_{hi}}, \mathbf{b_{lo}} \rangle]Q \end{aligned}$$

Commitment compression

Define $P_k \leftarrow P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle]Q$ – current commitment to vectors $\mathbf{a}, \mathbf{b}$ and define P_{k-1} using compressed vectors to have the same form as P_k, but in new basis $(\mathbf{G}^{(k-1)}, \mathbf{H}^{(k-1)})$:

$$P_{k-1} = \langle \mathbf{a}^{(k-1)}, \mathbf{G}^{(k-1)} \rangle + \langle \mathbf{b}^{(k-1)}, \mathbf{H}^{(k-1)} \rangle + [\langle \mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)} \rangle]Q$$

Substituting compressed vectors and applying bilinearity property of inner product we get:

$$\begin{aligned} P_{k-1} = &\langle \mathbf{a_{lo}}, \mathbf{G_{lo}} \rangle + \langle \mathbf{a_{hi}}, \mathbf{G_{hi}} \rangle && + u_k^2 \langle \mathbf{a_{lo}}, \mathbf{G_{hi}} \rangle + u_k^{-2} \langle \mathbf{a_{hi}}, \mathbf{G_{lo}} \rangle + \\ &\langle \mathbf{b_{lo}}, \mathbf{H_{lo}} \rangle + \langle \mathbf{b_{hi}}, \mathbf{H_{hi}} \rangle && + u_k^2 \langle \mathbf{b_{hi}}, \mathbf{H_{lo}} \rangle + u_k^{-2} \langle \mathbf{b_{lo}}, \mathbf{H_{hi}} \rangle + \\ &[\langle \mathbf{a_{lo}}, \mathbf{b_{lo}} \rangle + \langle \mathbf{a_{hi}}, \mathbf{b_{hi}} \rangle]Q && + [u_k^2 \langle \mathbf{a_{lo}}, \mathbf{b_{hi}} \rangle + u_k^{-2} \langle \mathbf{a_{hi}}, \mathbf{b_{lo}} \rangle]Q \end{aligned}$$

The first two columns precisely represent current commitment P_k, for the last two columns we define L_k, R_k as commitments to cross terms.

Commitment compression

So that we represent new commitment P_{k-1} from the old one P_k and cross terms L_k, R_k:

$$
\begin{aligned}
P_{k-1} &= P_k + [u_k^2]L_k + [u_k^{-2}]R_k \\
L_k &= \langle \mathbf{a_{lo}}, \mathbf{G_{hi}} \rangle + \langle \mathbf{b_{hi}}, \mathbf{H_{lo}} \rangle + [\langle \mathbf{a_{lo}}, \mathbf{b_{hi}} \rangle]Q \\
R_k &= \langle \mathbf{a_{hi}}, \mathbf{G_{lo}} \rangle + \langle \mathbf{b_{lo}}, \mathbf{H_{hi}} \rangle + [\langle \mathbf{a_{hi}}, \mathbf{b_{lo}} \rangle]Q
\end{aligned}
$$

Commitment compression

So that we represent new commitment P_{k-1} from the old one P_k and cross terms L_k, R_k:

$$
\begin{aligned}
P_{k-1} &= P_k + [u_k^2]L_k + [u_k^{-2}]R_k \\
L_k &= \langle \mathbf{a_{lo}}, \mathbf{G_{hi}} \rangle + \langle \mathbf{b_{hi}}, \mathbf{H_{lo}} \rangle + [\langle \mathbf{a_{lo}}, \mathbf{b_{hi}} \rangle]Q \\
R_k &= \langle \mathbf{a_{hi}}, \mathbf{G_{lo}} \rangle + \langle \mathbf{b_{lo}}, \mathbf{H_{hi}} \rangle + [\langle \mathbf{a_{hi}}, \mathbf{b_{lo}} \rangle]Q
\end{aligned}
$$

So the basic logic of compression step:

- Verifier draws challenge $u_k \leftarrow\!\!\$\ \mathbb{F}_p$ and sends it to prover

Commitment compression

So that we represent new commitment P_{k-1} from the old one P_k and cross terms L_k, R_k:

$$
\begin{aligned}
P_{k-1} &= P_k + [u_k^2]L_k + [u_k^{-2}]R_k \\
L_k &= \langle \mathbf{a_{lo}}, \mathbf{G_{hi}} \rangle + \langle \mathbf{b_{hi}}, \mathbf{H_{lo}} \rangle + [\langle \mathbf{a_{lo}}, \mathbf{b_{hi}} \rangle]Q \\
R_k &= \langle \mathbf{a_{hi}}, \mathbf{G_{lo}} \rangle + \langle \mathbf{b_{lo}}, \mathbf{H_{hi}} \rangle + [\langle \mathbf{a_{hi}}, \mathbf{b_{lo}} \rangle]Q
\end{aligned}
$$

So the basic logic of compression step:

- Verifier draws challenge $u_k \leftarrow\!\!\$\ \mathbb{F}_p$ and sends it to prover
- Prover computes $\mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)}$ and L_k, R_k and sends them to verifier

Commitment compression

So that we represent new commitment P_{k-1} from the old one P_k and cross terms L_k, R_k:

$$P_{k-1} = P_k + [u_k^2]L_k + [u_k^{-2}]R_k$$
$$L_k = \langle \mathbf{a_{lo}}, \mathbf{G_{hi}} \rangle + \langle \mathbf{b_{hi}}, \mathbf{H_{lo}} \rangle + [\langle \mathbf{a_{lo}}, \mathbf{b_{hi}} \rangle]Q$$
$$R_k = \langle \mathbf{a_{hi}}, \mathbf{G_{lo}} \rangle + \langle \mathbf{b_{lo}}, \mathbf{H_{hi}} \rangle + [\langle \mathbf{a_{hi}}, \mathbf{b_{lo}} \rangle]Q$$

So the basic logic of compression step:

- Verifier draws challenge $u_k \leftarrow\!\!\$\ \mathbb{F}_p$ and sends it to prover
- Prover computes $\mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)}$ and L_k, R_k and sends them to verifier
- Verifier reconstructs P_{k-1} using $\mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)}$ and checks:

$$P_{k-1} = P_k + [u_k^2]L_k + [u_k^{-2}]R_k$$

Recursive compression

Remark

We wish not to send $\mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)}$ directly as this is inefficient due to still linear sizes, instead we apply recursion to compress these vectors to just one element.

Recursive compression

Remark

We wish not to send $\mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)}$ directly as this is inefficient due to still linear sizes, instead we apply recursion to compress these vectors to just one element.

Here we come up with some kind of statement compression algorithm reducing size of all vectors in half per compression step. Repeating compression algorithm k times we end up with sending vectors $\mathbf{a}^{(0)}, \mathbf{b}^{(0)}$ each of length one and P_0 containing all accumulated cross-terms:

$$P_0 = [a_1^{(0)}]G_1^{(0)} + [b_1^{(0)}]H_1^{(0)} + [a_1^{(0)}b_1^{(0)}]Q$$

$$P_0 = P_k + \sum_{i=1}^{k}([u_i^2]L_i + [u_i^{-2}]R_i)$$

Recursive compression

Remark

We wish not to send $\mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)}$ directly as this is inefficient due to still linear sizes, instead we apply recursion to compress these vectors to just one element.

Here we come up with some kind of statement compression algorithm reducing size of all vectors in half per compression step. Repeating compression algorithm k times we end up with sending vectors $\mathbf{a}^{(0)}, \mathbf{b}^{(0)}$ each of length one and P_0 containing all accumulated cross-terms:

$$P_0 = [a_1^{(0)}]G_1^{(0)} + [b_1^{(0)}]H_1^{(0)} + [a_1^{(0)} b_1^{(0)}]Q$$

$$P_0 = P_k + \sum_{i=1}^{k}([u_i^2]L_i + [u_i^{-2}]R_i)$$

Verifier compares these P_0s and asserts inner-product correctness.

Inner-product argument protocol

Here we describe the **inner-product argument** protocol Π'_{ip} between prover $\mathcal{P}$ and verifier $\mathcal{V}$ for relation $\mathcal{R}'_{ip} = \{(\mathbf{G}, \mathbf{H}, Q, P'; \mathbf{a}, \mathbf{b}) : P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle] Q\}$ from scratch.

- Prover $\mathcal{P}$ sets

$$(k, \mathbf{a}^{(k)}, \mathbf{b}^{(k)}, \mathbf{G}^{(k)}, \mathbf{H}^{(k)}, P_k) \leftarrow (d, \mathbf{a}, \mathbf{b}, \mathbf{G}, \mathbf{H}, P')$$

Inner-product argument protocol

Here we describe the **inner-product argument** protocol Π'_{ip} between prover $\mathcal{P}$ and verifier $\mathcal{V}$ for relation $\mathcal{R}'_{ip} = \{(\mathbf{G}, \mathbf{H}, Q, P'; \mathbf{a}, \mathbf{b}) : P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle] Q\}$ from scratch.

- Prover $\mathcal{P}$ sets

$$(k, \mathbf{a}^{(k)}, \mathbf{b}^{(k)}, \mathbf{G}^{(k)}, \mathbf{H}^{(k)}, P_k) \leftarrow (d, \mathbf{a}, \mathbf{b}, \mathbf{G}, \mathbf{H}, P')$$

- Verifier $\mathcal{V}$ sets

$$(k, \mathbf{G}^{(k)}, \mathbf{H}^{(k)}, P_k) \leftarrow (d, \mathbf{G}, \mathbf{H}, P')$$

Inner-product argument protocol

Here we describe the **inner-product argument** protocol Π'_{ip} between prover $\mathcal{P}$ and verifier $\mathcal{V}$ for relation $\mathcal{R}'_{ip} = \{(\mathbf{G}, \mathbf{H}, Q, P'; \mathbf{a}, \mathbf{b}) : P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle] Q\}$ from scratch.

- Prover $\mathcal{P}$ sets

$$(k, \mathbf{a}^{(k)}, \mathbf{b}^{(k)}, \mathbf{G}^{(k)}, \mathbf{H}^{(k)}, P_k) \leftarrow (d, \mathbf{a}, \mathbf{b}, \mathbf{G}, \mathbf{H}, P')$$

- Verifier $\mathcal{V}$ sets

$$(k, \mathbf{G}^{(k)}, \mathbf{H}^{(k)}, P_k) \leftarrow (d, \mathbf{G}, \mathbf{H}, P')$$

- While $k > 0$ the parties engage in the **compression step protocol**

Compression step protocol

- Prover $\mathcal{P}$ computes and sends to $\mathcal{V}$

$$L_k = \langle \mathbf{a_{lo}}^{(k)}, \mathbf{G_{hi}}^{(k)} \rangle + \langle \mathbf{b_{hi}}^{(k)}, \mathbf{H_{lo}}^{(k)} \rangle + [\langle \mathbf{a_{lo}}^{(k)}, \mathbf{b_{hi}}^{(k)} \rangle] Q$$

$$R_k = \langle \mathbf{a_{hi}}^{(k)}, \mathbf{G_{lo}}^{(k)} \rangle + \langle \mathbf{b_{lo}}^{(k)}, \mathbf{H_{hi}}^{(k)} \rangle + [\langle \mathbf{a_{hi}}^{(k)}, \mathbf{b_{lo}}^{(k)} \rangle] Q$$

Compression step protocol

- Prover $\mathcal{P}$ computes and sends to $\mathcal{V}$

$$L_k = \langle \mathbf{a_{lo}}^{(k)}, \mathbf{G_{hi}}^{(k)} \rangle + \langle \mathbf{b_{hi}}^{(k)}, \mathbf{H_{lo}}^{(k)} \rangle + [\langle \mathbf{a_{lo}}^{(k)}, \mathbf{b_{hi}}^{(k)} \rangle] Q$$

$$R_k = \langle \mathbf{a_{hi}}^{(k)}, \mathbf{G_{lo}}^{(k)} \rangle + \langle \mathbf{b_{lo}}^{(k)}, \mathbf{H_{hi}}^{(k)} \rangle + [\langle \mathbf{a_{hi}}^{(k)}, \mathbf{b_{lo}}^{(k)} \rangle] Q$$

- $\mathcal{V}$ draws challenge $u_k \leftarrow\!\$\ \mathbb{F}_p$ and sends it to $\mathcal{P}$

Compression step protocol

- Prover $\mathcal{P}$ computes and sends to $\mathcal{V}$

$$L_k = \langle \mathbf{a_{lo}}^{(k)}, \mathbf{G_{hi}}^{(k)} \rangle + \langle \mathbf{b_{hi}}^{(k)}, \mathbf{H_{lo}}^{(k)} \rangle + [\langle \mathbf{a_{lo}}^{(k)}, \mathbf{b_{hi}}^{(k)} \rangle] Q$$

$$R_k = \langle \mathbf{a_{hi}}^{(k)}, \mathbf{G_{lo}}^{(k)} \rangle + \langle \mathbf{b_{lo}}^{(k)}, \mathbf{H_{hi}}^{(k)} \rangle + [\langle \mathbf{a_{hi}}^{(k)}, \mathbf{b_{lo}}^{(k)} \rangle] Q$$

- $\mathcal{V}$ draws challenge $u_k \leftarrow\!\$\ \mathbb{F}_p$ and sends it to $\mathcal{P}$
- Both $\mathcal{P}$ and $\mathcal{V}$ compute:

$$\mathbf{G}^{(k-1)} = \mathbf{G_{lo}}^{(k)} \cdot u_k^{-1} + u_k \cdot \mathbf{G_{hi}}^{(k)}$$

$$\mathbf{H}^{(k-1)} = \mathbf{H_{lo}}^{(k)} \cdot u_k + u_k^{-1} \cdot \mathbf{H_{hi}}^{(k)}$$

Compression step protocol

- Prover $\mathcal{P}$ computes and sends to $\mathcal{V}$

$$L_k = \langle \mathbf{a_{lo}}^{(k)}, \mathbf{G_{hi}}^{(k)} \rangle + \langle \mathbf{b_{hi}}^{(k)}, \mathbf{H_{lo}}^{(k)} \rangle + [\langle \mathbf{a_{lo}}^{(k)}, \mathbf{b_{hi}}^{(k)} \rangle] Q$$
$$R_k = \langle \mathbf{a_{hi}}^{(k)}, \mathbf{G_{lo}}^{(k)} \rangle + \langle \mathbf{b_{lo}}^{(k)}, \mathbf{H_{hi}}^{(k)} \rangle + [\langle \mathbf{a_{hi}}^{(k)}, \mathbf{b_{lo}}^{(k)} \rangle] Q$$

- $\mathcal{V}$ draws challenge $u_k \leftarrow\!\$\ \mathbb{F}_p$ and sends it to $\mathcal{P}$
- Both $\mathcal{P}$ and $\mathcal{V}$ compute:

$$\mathbf{G}^{(k-1)} = \mathbf{G_{lo}}^{(k)} \cdot u_k^{-1} + u_k \cdot \mathbf{G_{hi}}^{(k)}$$
$$\mathbf{H}^{(k-1)} = \mathbf{H_{lo}}^{(k)} \cdot u_k + u_k^{-1} \cdot \mathbf{H_{hi}}^{(k)}$$

- $\mathcal{P}$ computes:

$$\mathbf{a}^{(k-1)} = \mathbf{a_{lo}}^{(k)} \cdot u_k + u_k^{-1} \cdot \mathbf{a_{hi}}^{(k)}$$
$$\mathbf{b}^{(k-1)} = \mathbf{b_{lo}}^{(k)} \cdot u_k^{-1} + u_k \cdot \mathbf{b_{hi}}^{(k)}$$

Final step

At the final step when $k = 0$ parties perform final check:

- Prover $\mathcal{P}$ sends $(a, b) \leftarrow (\mathbf{a}_1^{(0)}, \mathbf{b}_1^{(0)})$ to verifier $\mathcal{V}$

Final step

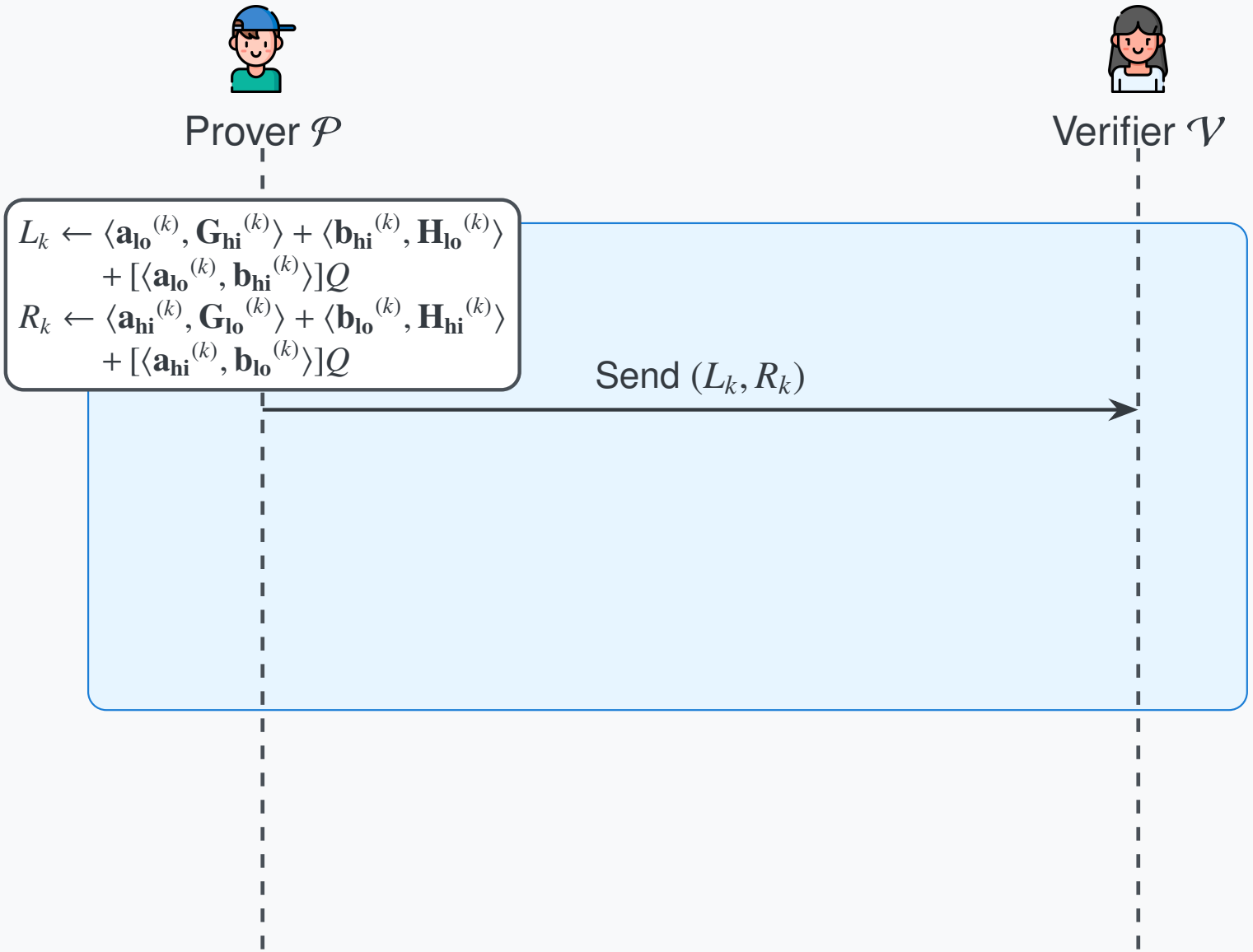
At the final step when $k = 0$ parties perform final check:

- Prover $\mathcal{P}$ sends $(a, b) \leftarrow (\mathbf{a}_1^{(0)}, \mathbf{b}_1^{(0)})$ to verifier $\mathcal{V}$
- Verifier performs final check:

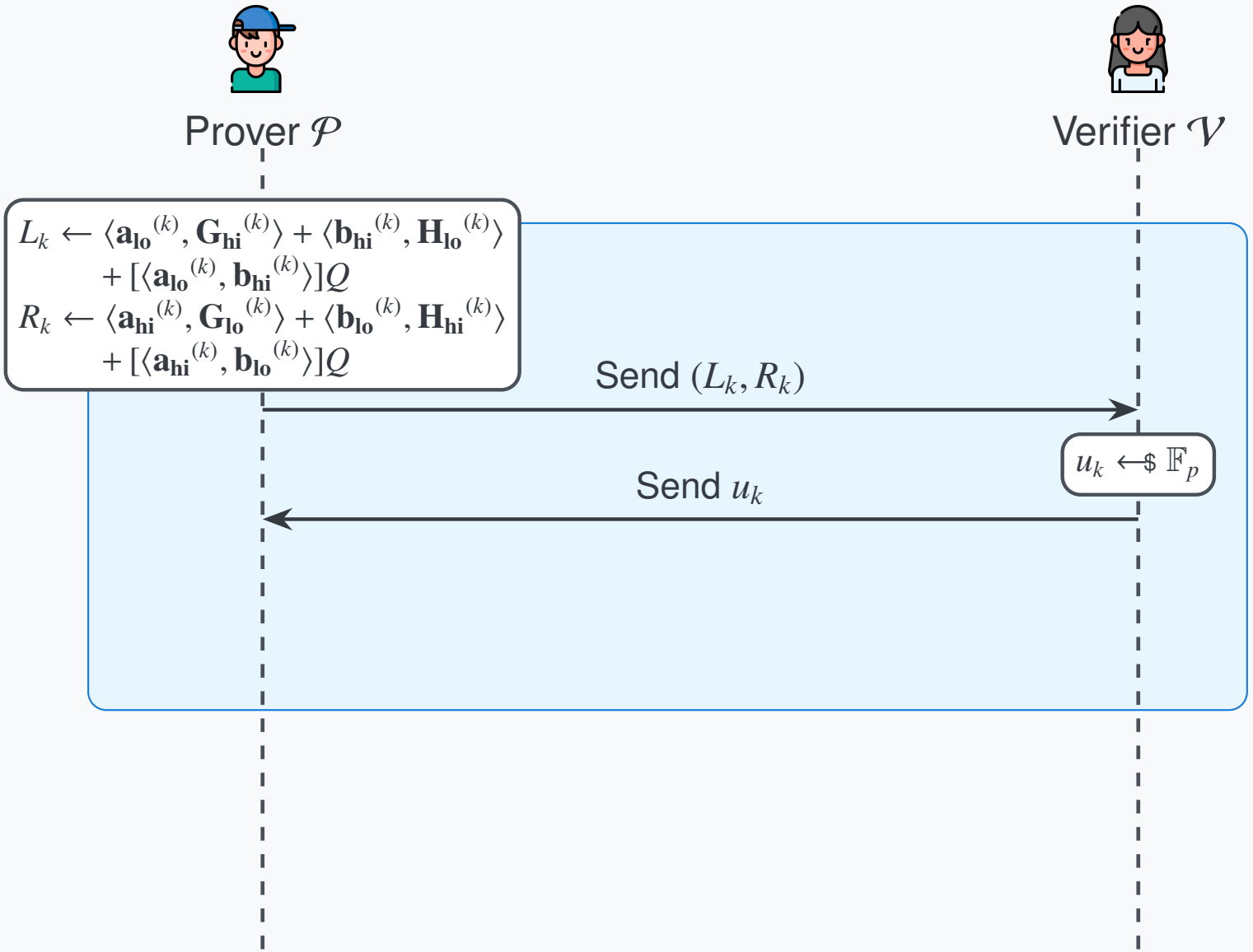
$$P' + \sum_{i=1}^{d} ([u_i^2]L_i + [u_i^{-2}]R_i) = [a]G_1^{(0)} + [b]H_1^{(0)} + [ab]Q$$

outputs **accept** if equality holds and **reject** otherwise.

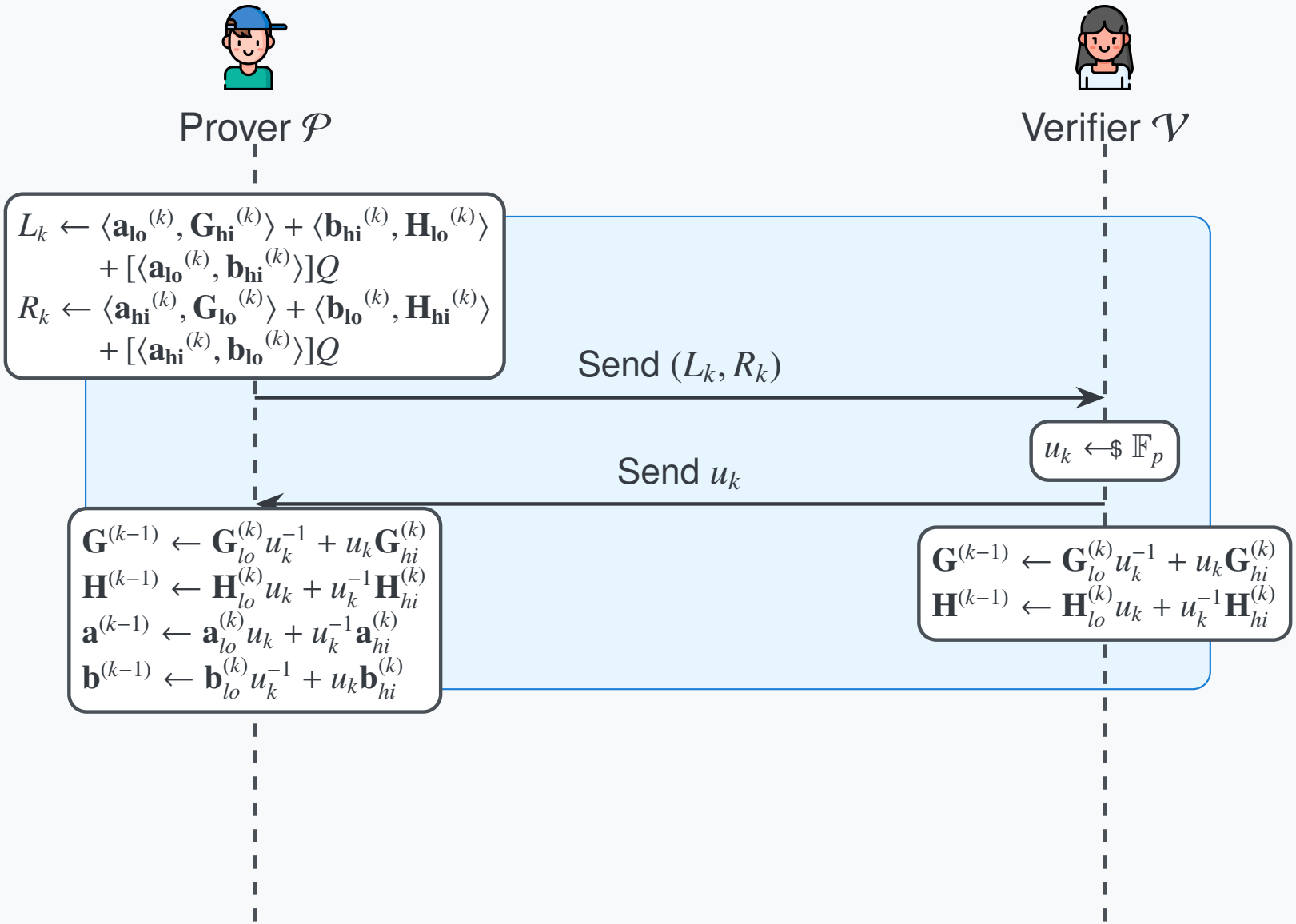
Inner-product argument: illustration



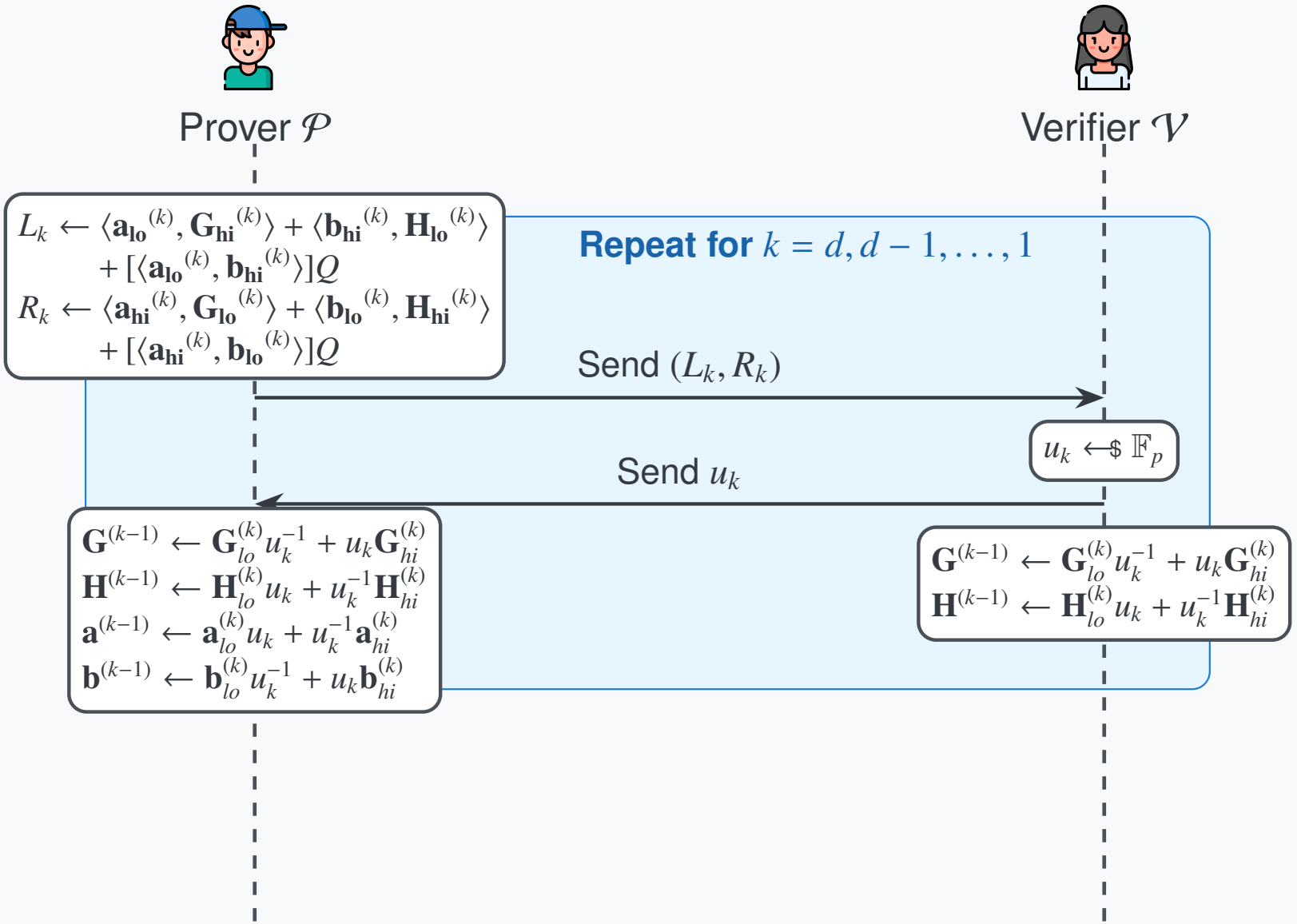
Inner-product argument: illustration



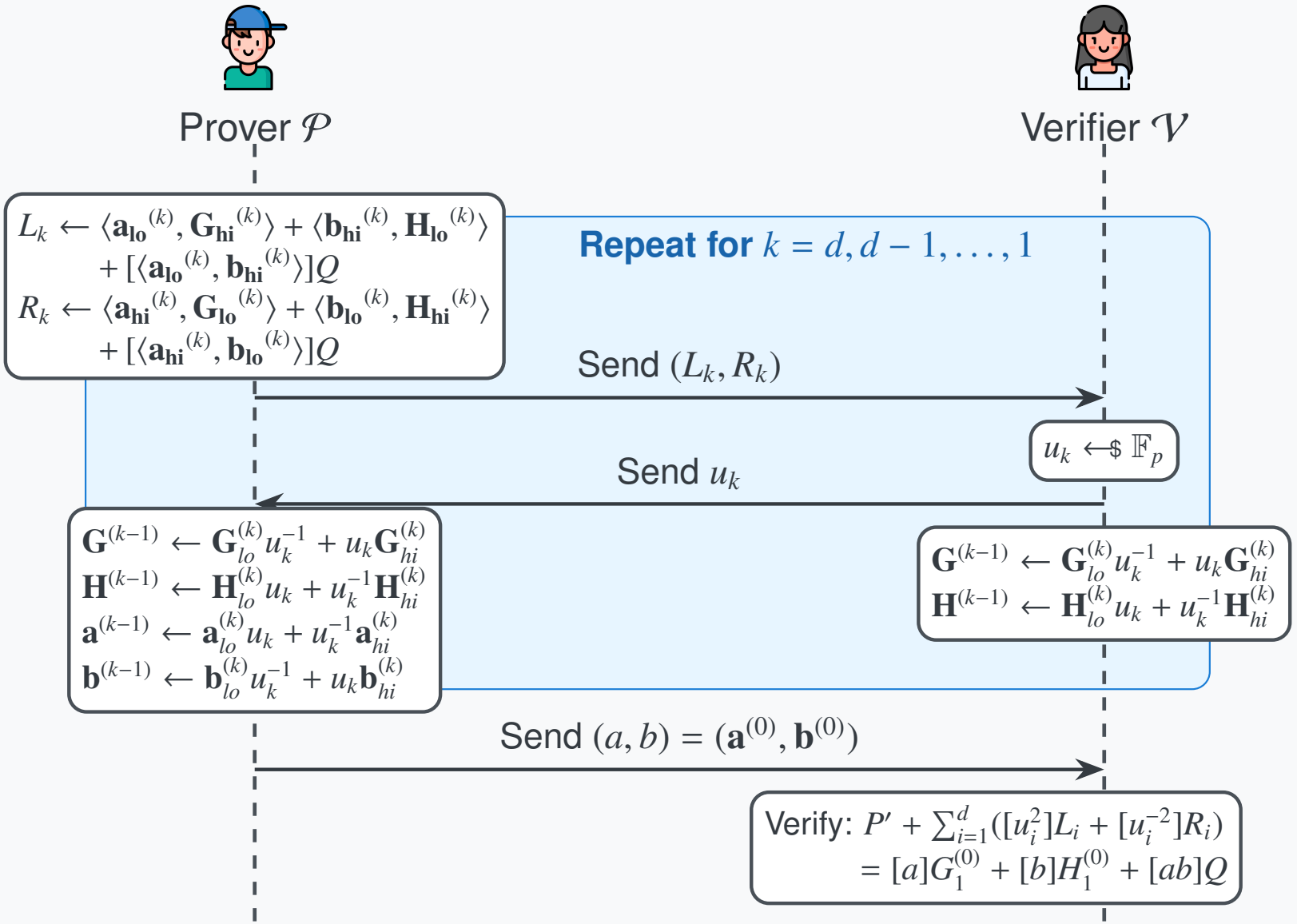
Inner-product argument: illustration



Inner-product argument: illustration



Inner-product argument: illustration



Inner-product argument: security & performance

Remark

Overall communication complexity of **inner-product argument** is $2 \log_2 n$ group elements plus 2 field elements so we come up with logarithmic proof size for our inner-product relation $\mathcal{R}_{ip}$.

Inner-product argument: security & performance

Remark

Overall communication complexity of **inner-product argument** is $2\log_2 n$ group elements plus 2 field elements so we come up with logarithmic proof size for our inner-product relation $\mathcal{R}_{ip}$.

Theorem (Inner-Product Argument)

The **inner-product argument** Π_{ip} for relation $\mathcal{R}_{ip}$ has *perfect completeness and statistical witness-extended emulation* for either extracting a non-trivial discrete logarithm relation between $\mathbf{G}, \mathbf{H}, Q$ or extracting valid witness $\mathbf{a}, \mathbf{b}$.

Inner-product argument: security & performance

Remark

Overall communication complexity of **inner-product argument** is $2\log_2 n$ group elements plus 2 field elements so we come up with logarithmic proof size for our inner-product relation $\mathcal{R}_{ip}$.

Theorem (Inner-Product Argument)

The **inner-product argument** Π_{ip} for relation $\mathcal{R}_{ip}$ has *perfect completeness and statistical witness-extended emulation* for either extracting a non-trivial discrete logarithm relation between $\mathbf{G}, \mathbf{H}, Q$ or extracting valid witness $\mathbf{a}, \mathbf{b}$.

Note

Zero-knowledge doesn't hold as if $n = 1$ then $\mathcal{P}$ sends witness pair a, b directly. We'll later compile efficient **inner-product argument** with zero-knowledge **zk-mul** protocol to achieve efficient zero-knowledge proofs for range proofs and arithmetic circuits.

What's next?


IPA
zk-mul
Range proofs &
Arithmetic circuits
@Petirep
JAKE-CLARK.TUMBLR
imgflip.com

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream